

## 1.1 Overview of Web Applications and Evolution of Web Architecture

World Wide Web (WWW), commonly known as the Web, is an information system enabling information to be shared over the Internet through simplified ways meant to appeal to users beyond IT specialists and hobbyists, as well as documents and other web resources to be accessed over the Internet according to specific rules, the Hypertext Transfer Protocol (HTTP).

A web application is a software program that runs in a web browser and uses the internet to send/receive data. Examples: Gmail, Facebook, E-commerce sites, Online banking dashboards, etc. Unlike traditional desktop applications, web apps don't need installation — users simply open a URL.

### Key Characteristics of Web Applications:

- Users interact through a browser using HTML, CSS, and JavaScript.
- Platform Independent
- Always Updated
- Client–Server Architecture
- Uses Internet Technologies

### How a Web Application Works (Simple Flow)

1. User enters a URL in browser
2. Browser sends HTTP/HTTPS request
3. Server receives request
4. Server processes → interacts with database
5. Server sends back response (HTML, CSS, JS or JSON data)
6. Browser renders the result on screen

### Types of Web Applications

#### 1. Static Web Apps

- Content does not change unless manually modified.
- Built with HTML/CSS.
- Example: basic portfolio sites.

#### 2. Dynamic Web Apps

- Content changes based on user input or database.
- Uses server-side languages like Node.js, PHP, Python (Django), Java.

#### 3. Single-Page Applications (SPA)

- Loads one HTML page, updates content dynamically using JS.
- Faster, app-like experience.
- Example: Gmail, Facebook

#### 4. Progressive Web Apps (PWA)

- Works offline
- Can be installed on home screen
- Example: Twitter Lite

#### 5. E-commerce Web Apps

- Dynamic product listings
- Payments, user accounts

### **Evolution of Web or Rise of the World Wide Web**

#### 1. The Early Web (Web 1.0) – Static Architecture

- Time Period: 1990s
- Style: Static websites
- Technologies: HTML, basic CSS, no JavaScript initially
- Characteristics:
  - Pages were static HTML files
  - No database, no user interaction
  - Read-only content (“Information Web”)
  - Server only served files on request
- Examples: Early personal websites, news page without login system, etc

#### 2. Web 2.0 – Dynamic Client–Server Architecture

- Time Period: 2000–2010
- Style: Dynamic, interactive web
- Technologies: JavaScript, PHP, ASP.NET, Java, MySQL
- Characteristics:
  - Pages generated dynamically using server-side languages
  - Databases introduced (MySQL, PostgreSQL)
  - Login systems, forms, user accounts Start of social media AJAX introduced → partial page updates
- Example: Early Facebook, Early Gmail, E-commerce sites, etc

#### 3. REST API + AJAX Era

- Time Period: 2010+
- Technologies: JSON, REST APIs, jQuery, Node.js, Django
- Characteristics:
  - Web became service-oriented
  - Frontend and backend separated
  - Data shared through JSON REST APIs
  - Frontend fetches data without reloading the page
- Example: Systems with same backend server serving websites, mobile, etc

#### **4. Single Page Applications (SPA)**

- Time Period: 2014+
- Technologies: React, Angular, Vue, Webpack
- Characteristics:
  - Only one HTML page loads
  - Everything else is rendered using JavaScript
  - API calls fetch data from server
  - Very fast and smooth user experience App-like behavior
- Example: Gmail, Facebook, Twitter, Drive

#### **5. Microservices + Cloud Architecture (Modern Web)**

- Time Period: Present
- Technologies: Docker, Kubernetes, Serverless, AWS, GCP, Microservices, GraphQL, gRPC
- Characteristics:
  - Large apps split into independent services
  - Highly scalable and fault-tolerant
  - Use of cloud platforms
  - Frontend may be SPA or mobile app
- Backend is a collection of small services communicating together

### **Types of Web Architecture**

#### **1. Monolithic Architecture:**

- The entire application (UI, business logic, data access) is built as a single, unified code base.
- Easy to develop initially, simple to deploy, easier debugging.
- Hard to scale (must scale the whole app, not just bottlenecks),
- A single bug can crash the whole system.
- Example: Wordpress CMS

#### **2. Microservices:**

- Breaking the application into small, independent services that communicate via APIs (REST or gRPC). Each service can have its own database and technology stack.
- Highly scalable, fault-tolerant (if one service fails, the app survives)
- Enables independent teams to work on different features.
- High complexity in DevOps, monitoring, and inter-service communication. ○ Example: Netflix, Amazon

### 3. Event-Driven Architecture (EDA):

- Services do not call each other directly (coupling). Instead, they emit "events"
- (e.g., "Order Placed") to a message broker (like Kafka or RabbitMQ), and
- other services listen and react.
- Extreme decoupling, real-time responsiveness, high throughput.
- Hard to trace the flow of logic; requires complex infrastructure.
- Example: LinkedIn, Netflix

### 4. Serverless (FaaS):

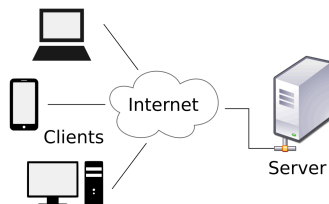
- Developers write individual functions (e.g., AWS Lambda) triggered by events.
- The cloud provider manages the servers entirely.
- Zero server management, pay-only-for-execution.
- Cold start problem (to boot up containers), vendor lock-in, distributed complexity
- Example: Netflix, Amazon

## 1.2 Client-server architecture, HTTP, HTTPS, URLs, DNS, web browsers

### Client–Server Architecture

Client–Server Architecture is a network model where two main components interact:

1. Client
  - a. The device or software that requests services or resources.
  - b. Processes and displays responses
  - c. User interface and presentation layer
  - d. Examples: Web browsers (Chrome, Firefox), mobile apps, desktop applications.
2. Server
  - a. A powerful computer or software that stores data, processes logic, and provides services to clients.
  - b. Examples: Web servers, database servers, authentication servers.



### Simple Workflow of How it works?

1. The client sends a request to the server.  
Example: typing a URL → browser sends a request to the web server.
2. The server processes the request.  
It may fetch data, run logic, or validate information.

3. The server sends a response back to the client.
4. Example: HTML page, JSON data, image, video, etc.
5. The client displays or uses the response.

### Real-life Example

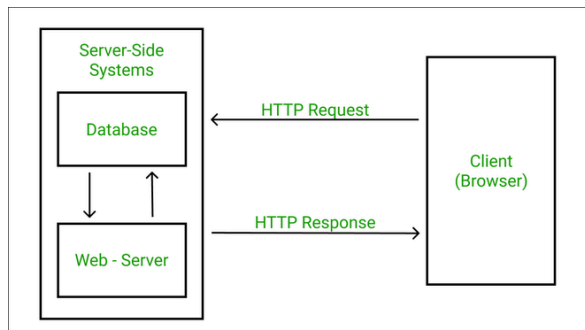
1. You open facebook.com
2. Browser (client) requests the page
3. Facebook server processes & sends back HTML/CSS/JS\
4. Browser displays the Facebook homepage.

### Advantages of Client–Server Architecture

- Easy to manage (data stored centrally)
- More secure than peer-to-peer
- Efficient resource sharing
- Supports many clients at once

### Disadvantages of Client–Server Architecture

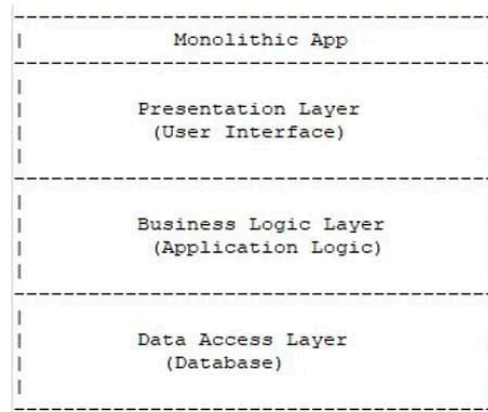
- If the server crashes → clients cannot access services
- Can become slow if overloaded
- Requires network connection



**Client-server architectures can be categorized into different tiers**

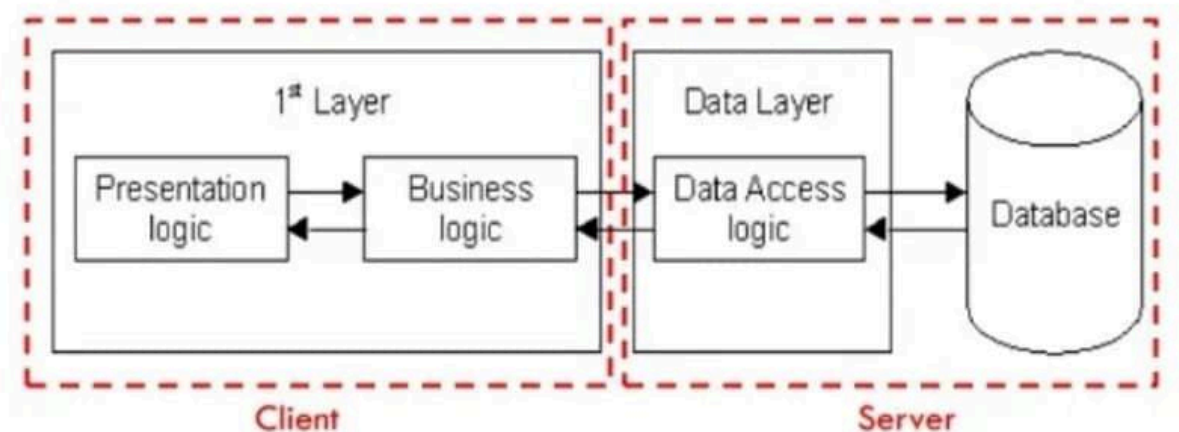
**1. Single-tier Architecture(Monolithic Architecture):**

- In a single-tier architecture, the entire application runs on a single machine or server.
- The user interface, application logic, and data storage are all combined into a single unit.
- This architecture is typically used for small-scale applications with limited complexity.



**2. Two-tier Architecture:**

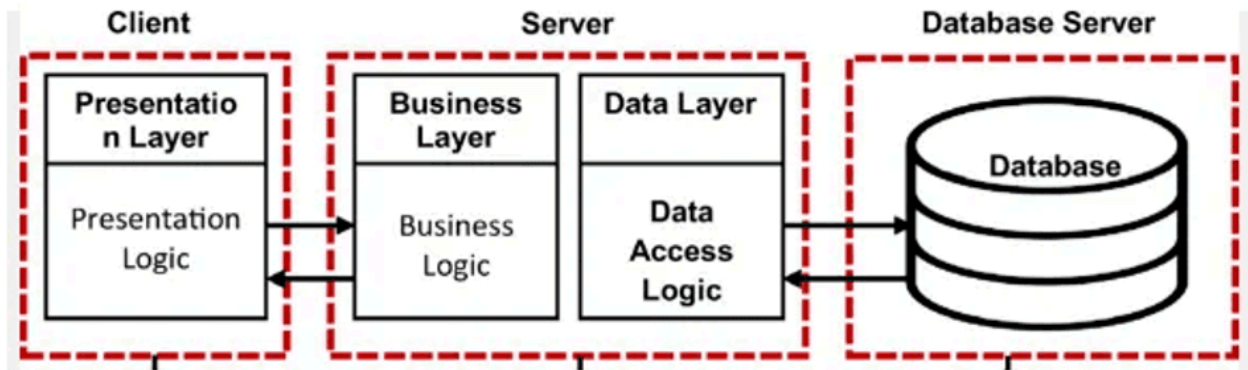
- In a two-tier architecture, the application is divided into two main tiers: client and server.
- The client tier is responsible for the user interface and user interactions.
- The server tier handles the application logic, data processing, and storage.
- Communication between the client and server occurs directly.



**3. Multi-tier Architecture:**

- This architecture divides the application into three or more tiers, each responsible for specific functionality.
- Common tiers include the presentation tier (client interface), application tier (business logic), and data tier (data storage and retrieval).

- Each additional tier represents a distinct set of responsibilities or services.
- Additional tiers can include caching layers, load balancers, message queues, microservices, etc.
- This architecture allows for better scalability, flexibility, and modularity, but also increases complexity.



**Fig: Three-tier Client-Server Architecture**

## HTTP/HTTPS

HTTP and HTTPS are **communication protocols** used by clients and servers to exchange information on the web. They define a set of rules for how **a client requests data** and **how a server responds**.

### HTTP

HTTP is a **stateless protocol** used for transferring webpages, images, videos, JSON data, etc., between a **client** and a **web server**.

Key Features:

- **Stateless:** Each request is independent; the server does not remember previous requests.
- **Plain text communication:** Data is not encrypted.
- **Uses port 80** by default.
- Faster but **less secure**.

Where HTTP is used?

- Non-sensitive websites
- Internal networks
- Public information pages

### HTTPS (HTTP Secure)

HTTPS is HTTP + TLS/SSL encryption.

Key Features

- Encrypted communication → safe from hackers
- Uses TLS/SSL certificates
- Uses port 443
- Ensures Confidentiality, Integrity, and Authentication

Where HTTPS is used?

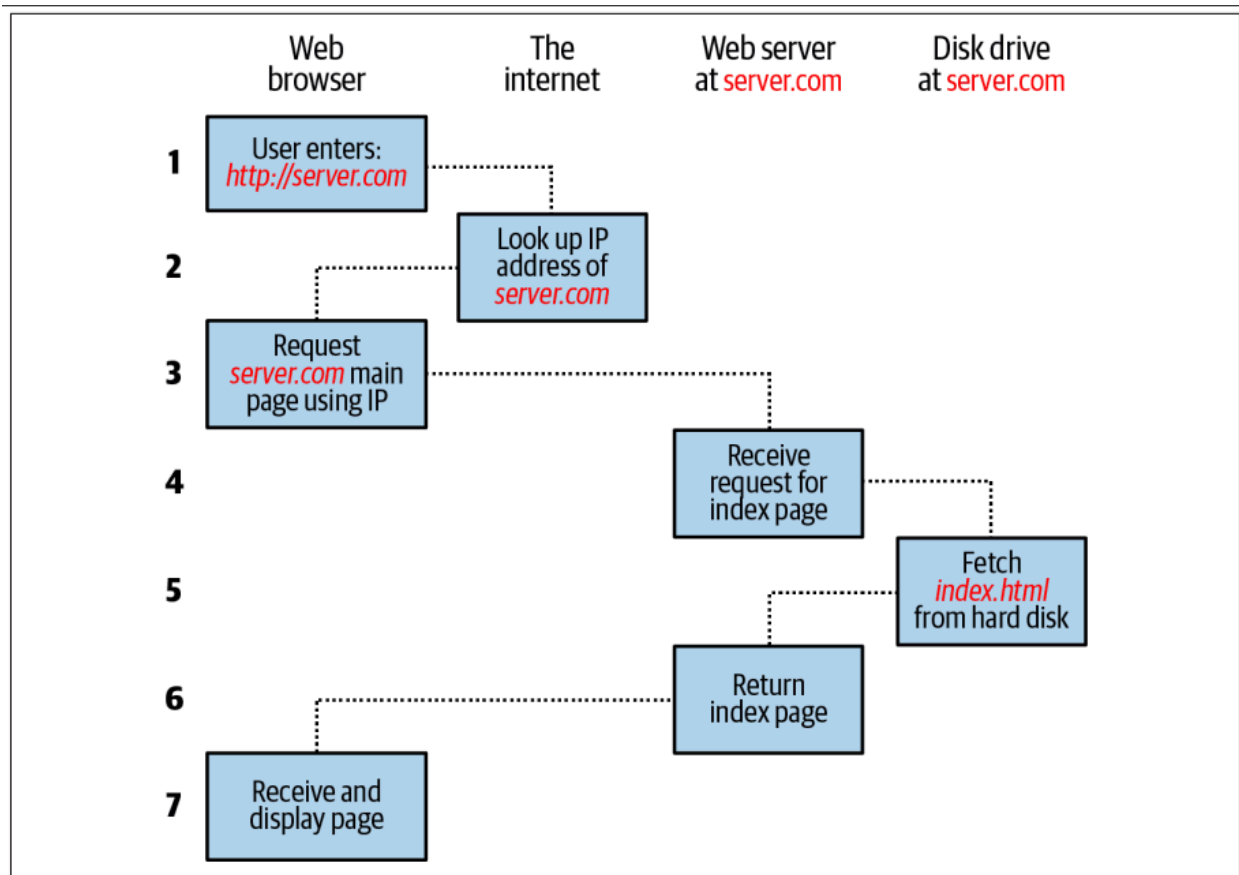
- Login pages
- Banking
- E-commerce
- APIs
- Any site handling user data

**How HTTP/HTTPS Works in Client–Server  
Flow Explanation:**



1. Client sends Request
  - Method (GET, POST, PUT, DELETE)
  - URL (example.com/page)
  - Headers (User-Agent, Accept, Authorization)
  - Body (only for POST/PUT)
2. Server receives and processes
  - Looks at the URL path
  - Runs backend logic
  - Retrieves data from database
3. Server sends Response
  - Status code (200 OK, 404 Not Found, 500 Server Error)
  - Headers (Content-Type, Cache-Control)
  - Body (HTML, JSON, Image, etc.)

**Try Yourself: Study the difference between HTTP and HTTPS in detail (including how is the security layer added in HTTPS in compared to HTTP)**



*Figure 1-1. The basic client/server request/response sequence*

## HTTP Request



1. Request Line: An HTTP request starts with a request line, which includes the following:

- Method: This indicates the action to be performed on the resource. Common methods include GET (retrieve a resource), POST (submit data to the server), PUT (update a resource), DELETE (remove a resource), and others.
- Request URL: The URL of the resource being requested.
- HTTP Version: The version of the HTTP protocol being used.

2. Request Headers: Following the request line are headers, which provide additional information about the request. This can include details about the client's browser, the types of responses that the client will accept, cookies, and more.

3. Blank Line: A blank line indicates the end of the headers section.

4. Request Body (Optional): Not all requests have a body. Bodies are typically included in POST or PUT requests, where you're sending data to the server (like form inputs or file uploads). The body contains the data being sent.

## HTTP Response

An HTTP response is what a server sends back to the client after receiving and processing an HTTP request. Here's a breakdown of the components of an HTTP response:

1. Status Line: This is the first line of the response and includes:

- **HTTP Version:** Indicates the HTTP protocol version used (e.g., HTTP/1.1).
- **Status Code:** A three-digit number that indicates the outcome of the request. Common status codes include 200 (OK, request succeeded), 404 (Not Found, the resource can't be found), 500 (Internal Server Error), etc.
- **Status Text:** A brief, human-readable explanation of the status code (e.g., OK, Not Found, Internal Server Error).

2. **Response Headers:** These are key-value pairs providing additional information about the response. They can include details such as the server type, content type, content length, caching policies, set cookies, and other metadata.

3. **Blank Line:** A blank line signifies the end of the header section.

4. **Response Body:** This part of the response contains the actual data or resource that the client requested. For example, it could be an HTML file, JSON data, an image, etc. In some cases, particularly when the response indicates an error (like a 404), the body might contain a message explaining the error.

### HTTP Methods (Verbs)

| Method | Purpose        | Description                             |
|--------|----------------|-----------------------------------------|
| GET    | Retrieve data  | Requests data from server, read-only    |
| POST   | Submit data    | Sends data to server, creates resources |
| PUT    | Update data    | Updates existing resource completely    |
| PATCH  | Partial update | Updates part of existing resource       |
| DELETE | Remove data    | Deletes specified resource              |
| HEAD   | Get headers    | Like GET but only retrieves headers     |

## HTTP Status Codes

### 1xx - Informational:

1. 100 Continue
  - a. The 100 Continue response indicates that the server has received the initial part of the request (the headers) and is ready for the client to proceed with sending the request body (the payload).
  - b. The client first sends only the request headers and includes a special header: Expect: 100-continue. This is the client asking, "Before I send this huge file, are the headers acceptable?"
2. 101 Switching Protocols
  - a. The most common use of the 101 status code is when establishing a WebSocket connection for real-time, bidirectional communication (like chat applications or live feeds).
  - b. The client sends an HTTP request (usually a GET) and includes the Upgrade header and the Connection: Upgrade header, specifying the desired new protocol (e.g., Upgrade: websocket).

### 2xx - Success:

1. 200 OK
2. 201 Created
3. 204 No Content

### 3xx - Redirection:

1. 301 Moved Permanently
2. 302 Found (Temporary Redirect)◦ A temporary redirect is ideal for short-lived content, such as directing traffic from a permanent URL (e.g., /sales) to a seasonal landing page (e.g., /holiday-sale-2025) for a few weeks.
3. 304 Not Modified (caching status)
  - a. It means the resource requested by the client has not been modified since the version specified by the client in its request headers (like If-Modified-Since or If-None-Match).

### 4xx - Client Errors:

1. 400 Bad Request (invalid user data)
2. 401 Unauthorized (not authenticated at all)
3. 403 Forbidden (authenticated but not like admin user)
4. 404 Not Found

## 5. 429 Too Many Requests

### 5xx - Server Errors:

#### 1. 500 Internal Server Error

- a. It is the server's way of saying, "Something went wrong, and I don't know exactly what."
- b. Common Causes:
  - i. Application Crashes
  - ii. web server lacks the necessary file permissions to run scripts or access databases.

#### 2. 502 Bad Gateway

- a. This error often occurs in architectures with multiple layers, such as:
- b. A web server (like Nginx or Apache) that acts as a reverse proxy receives a bad or incomplete response from the application server (where the code actually runs).
- c. A CDN (Content Delivery Network) receives an error from the origin server.

#### 3. 503 Service Unavailable

- a. The server is temporarily unable to handle the request due to temporary overloading or scheduled maintenance.
- b. The response often includes a Retry-After header, indicating how long the service will be unavailable (e.g., Retry-After: 3600 for one hour).

## URIs (Uniform Resource Identifier)

A **URI is an identifier** used to uniquely name or identify a resource on the internet. It may identify a resource **either by its location or by its name or both**.

**URI = URL + URN**

A URI can be:

- **URL** → identifies resource by *location*
- **URN** → identifies resource by *name* (not location)

### Example URIs

- <https://example.com/about>
- <mailto:admin@example.com>
- <tel:+97798000000>
- <urn:isbn:0451450523>

## URL (Uniform Resource Locator)

A URL is a type of URI that specifically tells where the resource is located and how to access it.

**URL = Protocol + Domain + Path + Query + Fragment**

**Eg: <https://www.example.com:443/products/laptop?id=10#reviews>**

Parts of URL:

|                                                                                      |                        |
|--------------------------------------------------------------------------------------|------------------------|
| <a href="https://www.example.com:443/products/item?id=5#reviews">https:</a>          | Protocol               |
| <a href="https://www.example.com:443/products/item?id=5#reviews">www.example.com</a> | Domain name            |
| <a href="https://www.example.com:443/products/item?id=5#reviews">443</a>             | Port number (optional) |
| <a href="https://www.example.com:443/products/item?id=5#reviews">/products/item</a>  | Path                   |
| <a href="https://www.example.com:443/products/item?id=5#reviews">?id=5</a>           | Query parameter        |
| <a href="https://www.example.com:443/products/item?id=5#reviews">#reviews</a>        | Fragment/section       |

## **DNS (Domain Name System)**

- DNS stands for Domain Name System, often called the “Phonebook of the Internet.”
- DNS converts human-readable domain names into IP addresses that computers can understand.
- Humans remember names like: google.com, facebook.com, openai.com, etc
- Computers only understand IP addresses like: 142.250.183.78 (IPv4), 2607:f8b0:4004:809::200e (IPv6)
- DNS translates human readable internet locations to computer understandable internet locations.
- When you type `www.facebook.com`, DNS finds the IP address of Facebook’s server.

## **DNS Servers (Types)**

**DNS Resolver (Client-side / ISP Resolver):** First stop when a browser requests a domain.

**Root DNS Server:** Knows all top-level domains (.com, .org, .edu, .np).

**TLD Server (Top Level Domain Server):** Handles domain extensions such as:  
.com, .org, .net, .edu, .np, .gov

**Authoritative DNS Server:** Contains the actual DNS records of the domain and Provides the final IP address.

## **DNS Lookup Process (Step-by-Step)**

### **Step 1: Browser Cache**

- Browser first checks its own DNS cache
- Example: Chrome, Firefox store recent lookups.
- If found → immediately returns the IP
- If not → go to next step

### **Step 2 : Operating System Cache**

- Your operating system (Windows/macOS/Linux) checks its DNS table.
- Command examples:
  - Windows: `ipconfig /displaydns`
  - Linux: `systemd-resolve --status`

- If found → OS returns IP
- If not → next step

### **Step 3 : DNS Resolver (ISP / Local Network)**

- The request goes to the DNS resolver of:
  - Your ISP (WorldLink, Ncell, Nepal Telecom, etc.)
  - Or a custom DNS (Google 8.8.8.8, Cloudflare 1.1.1.1)
- Resolver checks its cache
- If not found → it starts recursive lookup

### **Step 4 : Root DNS Server**

- Resolver queries a Root Server:
- Root servers know which TLD server to contact.
- Example:  
For example.com, root server says:  
"Go to the .com TLD server."

### **Step 5 : TLD Server**

- TLD server for .com returns:
- The Authoritative Name Server for that domain  
(E.g. ns1.exempldns.com)

### **Step 6 : Authoritative DNS Server**

- This server has the actual DNS records:
  - A record (IPv4)
  - AAAA record (IPv6)
  - CNAME
  - MX (mail)
  - TXT
  - NS records
- Authoritative server returns the IP address, like: 93.184.216.34

### **Step 7 : DNS Resolver returns IP to Browser**

- Resolver sends the IP to your OS → Browser.
- Both store in cache for future use (TTL).



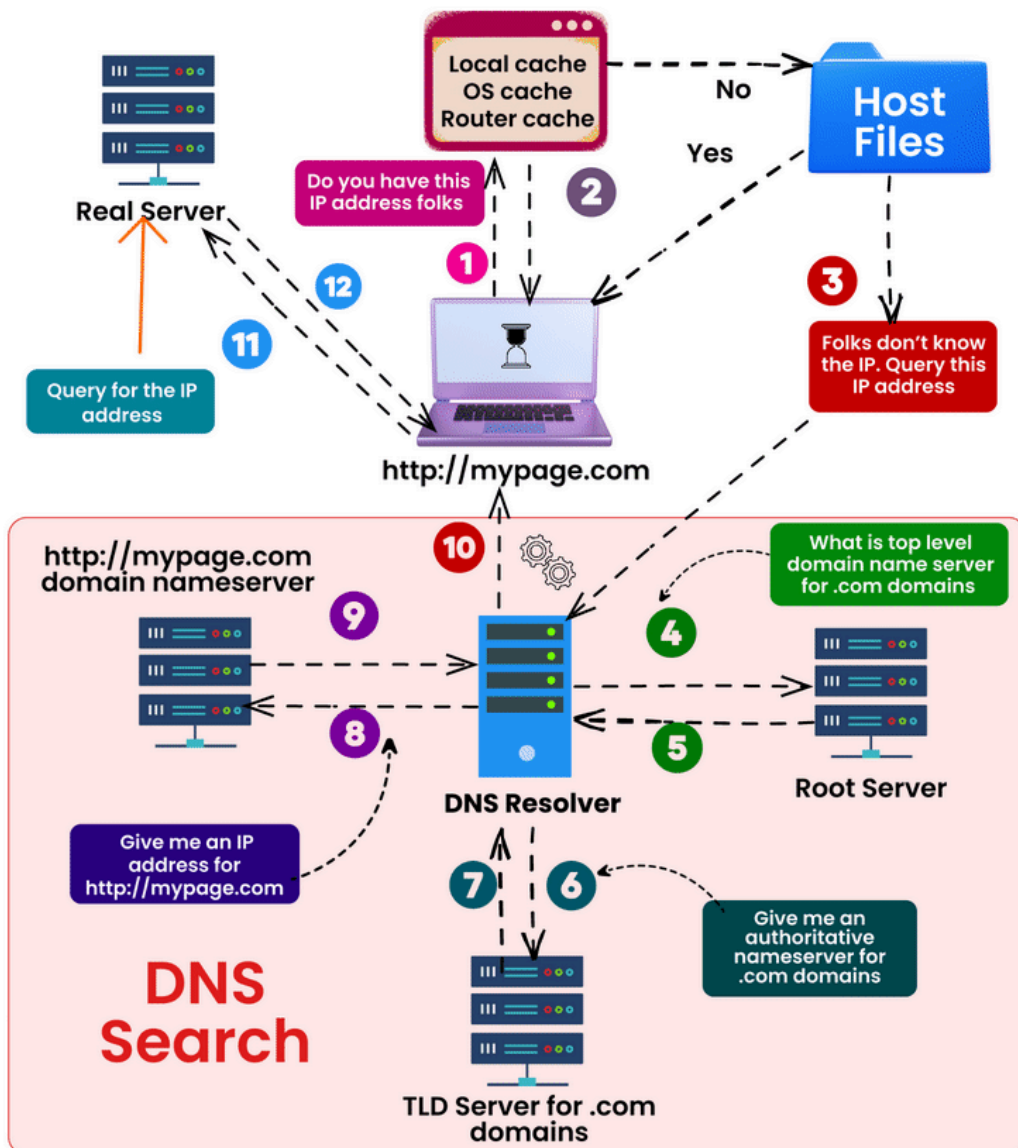
## Step 8 : Browser connects to Server

- Now browser makes an HTTP/HTTPS request to: 93.184.216.34

# HOW DNS WORKS



Brij Kishore Pandey  
DON'T FORGET TO SAVE



## **Search Engines**

A search engine is a software system designed to carry out web searches. They search the World Wide Web in a systematic way for particular information specified in a textual web search query.

### **How Search Engines Work**

1. Crawling:
  - Search engines use bots (spiders/crawlers) to browse the web.
  - They follow links from one page to another to discover new content.
  
2. Indexing:
  - The discovered pages are analyzed and stored in a massive database (index).
  - Content is categorized based on keywords, freshness, and relevance.
  
3. Ranking:
  - When a user searches, the engine retrieves matching results.
  - Algorithms rank these results based on relevance, authority, and user signals (SEO).

### **Popular Search Engines**

- Google (Dominant market share)
- Bing (Microsoft)
- Yahoo
- DuckDuckGo (Privacy-focused)
- Baidu (China)
- Yandex (Russia)

## **Web Browsers**

A web browser is a software application that retrieves, presents, and navigates information resources on the World Wide Web.

### **Popular Web Browsers:**

- **Google Chrome**
- **Mozilla Firefox**
- **Safari**
- **Microsoft Edge**
- **Opera**
- **Brave**

## **Browser Components**

### **1. User Interface:**

- Address bar
- Back/forward buttons
- Bookmarks
- Tabs and windows

### **2. Browser Engine:**

- Acts as a bridge between the User Interface and the Rendering Engine.
- It handles inputs from the UI (like clicking "Back" or typing a URL) and commands the Rendering Engine to display the correct content.

### **3. Rendering Engine:**

- Displays requested content
- Parses HTML and CSS
- Examples: Blink (Chrome), Gecko (Firefox), WebKit (Safari)

### **4. Networking:**

- Handles HTTP/HTTPS requests
- Manages cookies
- Implements caching

### **5. JavaScript Engine:**

- Parses and executes JavaScript code
- Examples: V8 (Chrome), SpiderMonkey (Firefox)

### **6. Data Storage:**

- Cookies
- LocalStorage
- SessionStorage
- IndexedDB
- Cache API

## How Browsers Render Pages

1. When a browser receives a web page, it first parses the HTML to create the DOM (Document Object Model), which represents the structure of the content.
2. Simultaneously, it parses the CSS to build the CSSOM (CSS Object Model), defining the styles for each element.
3. These two trees are then combined into a Render Tree, which contains only the visible elements.
4. The browser then performs the Layout (or Reflow) step, calculating the exact position and size of each element on the screen.
5. Finally, the browser paints the pixels for each element onto the screen.
6. If the page has complex visual effects (like overlapping elements or 3D transforms), it uses a Composite step to layer these painted parts together correctly for the final display.

## Browser Features

1. Developer Tools:
  - a. Inspect HTML/CSS
  - b. Debug JavaScript
  - c. Monitor network activity
  - d. Analyze performance
2. Security Features:
  - a. Same-origin policy
  - b. HTTPS enforcement
  - c. Pop-up blockers
  - d. Phishing protection
3. Modern Capabilities:
  - a. Progressive Web App support
  - b. WebRTC for real-time communication
  - c. WebAssembly for high-performance code allowing high-performance languages (C++, Rust) to run in the browser for demanding apps like games or video editors.
  - d. Geolocation API
  - e. Notifications API

## HTML and CSS Basics

Web development has three main building blocks:

- HTML:       Structure
- CSS:         Styling
- JS:          Behaviour

### HTML (HyperText Markup Language)

HTML is the backbone of a webpage.

It defines the **structure** and **content** — what you see on the screen.

#### What HTML Does

- Creates headings, paragraphs, images, links
- Defines page layout
- Adds forms, tables, lists
- Provides structure for CSS and JavaScript

### CSS (Cascading Style Sheets)

While HTML decides *what* appears, CSS controls *how it looks*.

#### What CSS Does:

- Colors, fonts, and backgrounds
- Spacing (margin, padding)
- Layouts (Flexbox, Grid)
- Animations and transitions
- Responsive design for mobile screens

## JavaScript (JS)

JavaScript adds interactivity and behavior to web pages. It makes the webpage *dynamic*, rather than static.

### What JavaScript Does

- Form validation
- Show/hide elements
- Sliders, carousels, animations
- API calls and dynamic content
- Logic and functionality

### HTML + CSS + JS Combined

```
<!DOCTYPE html>
<html>
<head>

    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>My Webpage</title>
    <link rel="icon" href="images/favicon.ico" />
    <style>
        h1 { color: green; }
    </style>
</head>
<body>
    <h1>Welcome to my webpage</h1>
    <button id="btn">Click Me</button>
    <script>
        document.getElementById("btn").onclick = () => {
            alert("Welcome to Web Development!");
        };
    </script>
</body>
</html>
```

### Structure Breakdown:

- **<!DOCTYPE html>** - Declares HTML5 document type
- **<html lang="en">** - Root element with language attribute
- **<head>** - Contains metadata (not visible on page)

- **<meta charset="UTF-8"> - Character encoding**
  - **<meta name="viewport" ...> - Responsive design settings**
  - **<title> - Page title shown in browser tab**
- **<body> - Contains all visible content**
- **<style> - Contains CSS styles**
- **<link> - Links to external resources**

## 1. HTML Syntax

- HTML (HyperText Markup Language) uses \*tags\* to structure webpages.

Example:

```
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello World</h1>
  </body>
</html>
```

- Tags come in pairs: <tag> content </tag>

## 2. HTML Tags

Common tags:

- <h1> to <h6> : Headings
- <p> : Paragraph
- <br> : Line break
- <hr> : Horizontal line
- <div> : Division/section
- <span> : Inline container

Example:

```
<p>This is a paragraph.</p>
```

## 3. HTML Attributes

Attributes give additional information to tags.

Example:

```

```



Common attributes:

- id
- class
- src
- href
- style
- alt

#### 4. HTML Forms & Inputs

Forms collect user input.

Example:

```
<form action="/submit" method="POST">  
  
<label>Name:</label>  
  
<input type="text" name="username" />  
  
<input type="submit" value="Submit" />  
  
</form>
```

Common input types:

- text, - number, - email, - password, - radio, - checkbox, - file, - submit

## 5. HTML Tables

Tables represent structured data.

Example:

```
<table border="1">
  <caption>
    Student Record
  </caption>
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ram</td>
      <td>20</td>
    </tr>
  </tbody>
</table>
```

Table Structure:

- <caption>: Title for the table
- <table>: Container for the entire table
- <thead>: Table header section
- <tbody>: Table body section
- <tfoot>: Table footer section
- <tr>: Table row
- <th>: Table header cell
- <td>: Table data cell

## 6. HTML Lists

Ordered list (numbered):

```
<ol>  
    <li>Apple</li>  
    <li>Banana</li>  
</ol>
```

Unordered list (bulleted):

```
<ul>  
    <li>Dog</li>  
    <li>Cat</li>  
</ul>
```

Description list:

```
<dl>  
    <dt>CPU</dt>  
    <dd>Central Processing Unit</dd>  
</dl>
```

## 7. Multimedia in HTML

To embed images, audio, and video:

Image:

```

```

Audio:

```
<audio controls>
```

```
<source src="song.mp3" type="audio/mpeg">
```

```
</audio>
```

Video:

```
<video controls width="400">
```

```
<source src="video.mp4" type="video/mp4">
```

```
</video>
```

## 8. Semantic HTML5 Elements

Semantic tags provide meaning and improve accessibility.

Common semantic tags:

<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>, <figure>, <figcaption>

Example:

```
<article>
```

```
<h2>News Title</h2>
```

```
<p>News description goes here...</p>
```

```
</article>
```

## 9. Forms in HTML

### Input Types

HTML forms support various input types for different data collection needs.

Syntax:

```
<form name="loginForm" >
    <input type="email" placeholder="email input field" />
    <br /><br />
    <input type="password" placeholder="password input field" />
    <br /><br />
    <button type="submit">Submit</button>
</form>
```

Different types supported by input field:

- text: Basic text input with maxlength attribute
- number: Numeric input with step attribute for increments
- checkbox: Can be checked by default
- password: Hides input text
- email: Validates email format
- submit/reset: Form control buttons
- radio: Grouped by name attribute, only one selectable per group
- date/time: Date and time pickers
- color: Color picker
- file: File upload
- search: Search input field

## 10. Iframe

The <iframe> tag specifies an inline frame, which is used to embed another document within the current HTML document.

**<iframe**

**src="**<https://www.example.com>**"**

**title="Example Website"**

**width="600"**

**height="400"**

**></iframe>**

Attributes:

- src: URL of the document to embed
- title: Description of the embedded content
- width/height: Dimensions of the iframe

## CSS Basics

- CSS (Cascading Style Sheets) is a style sheet language used for describing the look and formatting of a document written in a markup language.
- CSS is used to style and layout web pages, making it possible to create visually appealing and responsive designs.
- It is used to separate the presentation of a document from its content, making it easier to manage and update the design of a website.
- CSS is a key technology of the World Wide Web, alongside HTML and JavaScript.

Syntax:

```
selector {  
  
    property: value;  
  
}
```

## Adding CSS

CSS can be added to HTML documents in three different ways. Each method has its own use cases and priority levels. The Three Methods:

1. Inline CSS: Applied directly to HTML elements using the style attribute
2. Internal CSS: Written within <style> tags in the HTML document's <head> section
3. External CSS: Written in separate .css files and linked to the HTML document

### Priority Order (Specificity):

Inline CSS > Internal CSS / External CSS Priority of Internal CSS and External CSS depends on which comes last. Last style overrides previous one.

## CSS Selectors

Selectors target HTML elements. Key Selectors:

1. Element Selector: Selects elements by their tag name
2. Class Selector: Selects elements by their class attribute
3. ID Selector: Selects elements by their ID attribute

4. Universal Selector: Selects all elements on the page, selected with \*

5. Attribute Selector: Selects elements by their attribute, selected with

`input[type="text"]`

- IDs: Unique identifiers (should only be used once per page), selected with #
- Classes: Reusable identifiers (can be used multiple times), selected with .
- Priority: ID selectors have higher specificity than class selectors. Class selectors have

higher specificity than element selectors. Element selectors have higher specificity than attribute selectors.

## **CSS Colors**

CSS offers multiple ways to define colors, giving you flexibility in how you specify color values.

Color Methods:

1. Named Colors: Predefined color names (e.g., red, blue, yellow)
2. Hexadecimal: Using hex codes (e.g., #ff5733)
3. RGB: Using RGB values (e.g., rgb(255, 87, 51))
4. Other methods: HSL, RGBA, HSLA (for transparency)

## **Height and Width**

Height and width properties control the dimensions of elements. They can be set using various units including pixels, percentages, and other relative units. Key Concepts:

- Pixels (px): Absolute unit
- Percentages (%): Relative to parent element
- Parent-Child Relationships: Child elements inherit sizing context from parents



## **Borders**

Borders add visual boundaries around elements. You can control their width, style, and color independently for each side. Border Properties:

- border-width: Thickness of the border
- border-style: Style of the border (solid, dashed, dotted, double, etc.)
- border-color: Color of the border
- Individual sides: Can be styled separately using directional properties

## **Margin**

Margin creates space outside an element, pushing other elements away. It's the outermost layer of the CSS box model. Margin Properties:

- Individual sides: margin-top, margin-right, margin-bottom, margin-left
- Shorthand: margin can take 1-4 values
  - 1 value: all sides
  - 2 values: top/bottom, left/right
  - 4 values: top, right, bottom, left (clockwise)

## **Padding**

Padding creates space inside an element, between the content and the border. It's part of the element's total size.

Padding Properties:

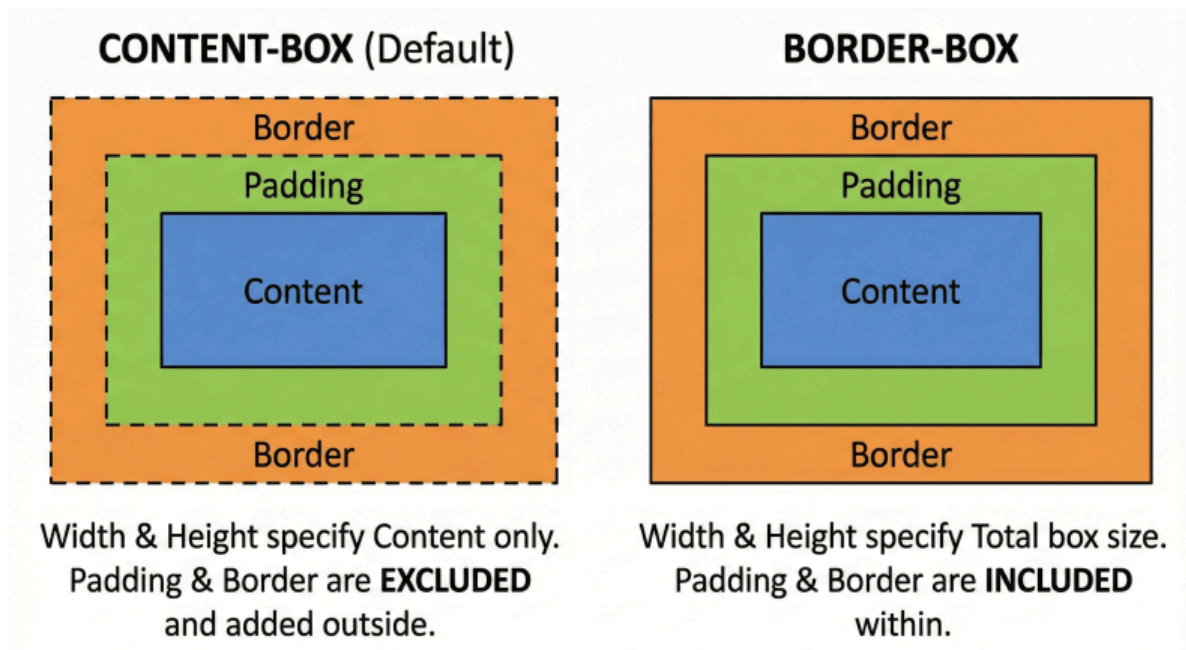
- Individual sides: padding-top, padding-right, padding-bottom, padding-left
- Shorthand: padding follows the same pattern as margin
- Effect: Increases the element's total size (unless using box-sizing: border-box)

## Box Sizing

Box-sizing determines how the total width and height of elements are calculated, affecting how padding and borders are included.

Box Model - The box model is the structure that defines the size and shape of an element. It is made up of four parts: content, padding, border, and margin. Box Sizing Values:

- Content-box (default): Width/height applies only to content
- Border-box: Width/height includes content, padding, and border



## Block vs Inline Elements

HTML elements are categorized into two main display types: Block and Inline elements. Understanding this distinction is crucial for proper layout design.

### Block Elements

- Take up full width available
- Start on a new line
- Can have width and height set
- Examples: `<div>`, `<h1>`-`<h6>`, `<p>`, `<li>`, `<hr>`

## Inline Elements

- Only take up as much width as necessary
- Don't start on a new line
- Width and height properties are ignored
- Examples: <a>, <span>, <strong>, <img>

## Inline-Block Elements

- Only take up as much width as necessary but Width and height properties can be set
- Combines features of both block and inline
- Examples: <button>, <input>

## Display Property

The display property allows you to change the default behavior of elements. You can make block elements behave like inline elements and vice versa.

### Display Values

- block: Makes element behave as block element
- inline: Makes element behave as inline element
- inline-block: Combines features of both (flows inline but accepts width/height)
- none: Hides the element completely

## Border Radius

The border-radius property creates rounded corners on elements. It's essential for modern web design and can transform sharp rectangular elements into circles, ovals, or elements with custom rounded corners.

### Border Radius Syntax

- Single value: border-radius: 10px (all corners)
- Four values: border-radius: 10px 20px 30px 40px (top-left, top-right, bottom-right, bottom-left)

bottom-right, bottom-left)

- Individual corners: border-top-left-radius, border-top-right-radius, etc.

## **Pseudo Classes**

Pseudo-classes target elements in specific states or positions, adding interactivity and dynamic styling.

### **Common Pseudo-Classes:**

- :hover: Mouse over state
- :active: Click state
- :focus: Input focus state
- :visited: Visited links
- :first-child, :last-child: Position-based
- :nth-child(): Nth position
- :valid, :invalid: Form validation states
- input:focus:invalid: Combine

### **Advanced Selectors:**

- :nth-child(2n): Every even child
- :nth-child(odd): Every odd child
- :checked + span: Sibling element after checked checkbox

## **Transitions**

CSS transitions create smooth animations between property changes, enhancing user experience with fluid motion.

### **Transition Properties:**

- transition-property: Which property to animate
- transition-duration: How long the animation takes
- transition-timing-function: Animation curve
- transition-delay: Delay before animation starts

## CSS Framework : Bootstrap

Bootstrap is the most popular CSS framework for building **responsive**, **mobile-first**, and **fast** web pages. Bootstrap is an open-source front-end framework used to quickly design websites using **pre-built CSS classes**, **components**, and **JavaScript plugins**.

Advantages of Bootstrap:

- Ready-made UI components
- Mobile-first responsive grid system
- Saves time
- Easy to customize
- Flexible and consistent UI
- Works well for beginners & advanced users

Setup

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
      integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script
      src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
      integrity="sha384-FKYoEForCGlyvwx9Hj09JcYn3nv7wiPV1z7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous">
    </script>
  </body>
</html>
```